

Universidad Autónoma de Chihuahua
Facultad de Ingeniería
Grupo 5K2

Actividad 1: Configuración de un clúster MongoDB con Docker y sharding

Bases de Datos Avanzadas

Kevin Andrés Rosales Rodríguez - 385814

Edwin Noe Zaragoza Alvarado - 385609

Manuel Ramirez Contreras- 385706

29/10/26

Objetivo

Implementar una base de datos distribuida con MongoDB sobre Docker utilizando un servidor de configuración, tres shards y un router mongos. Definir la estructura de Scott, elegir la shard key de empleados y realizar operaciones CRUD desde mongos, documentando la configuración y el reparto con evidencias de ejecución.

Introducción

MongoDB usa sharding para dividir una colección entre varios shards, en esta práctica cada shard almacena parte de los documentos, mientras que mongosh recibe las operaciones del cliente y las dirige al destino correspondiente, mientras ell servidor de configuración guarda los metadatos que describen la ubicación de los rangos de datos.

La replicación permite mantener copias dentro de un replica set. En esta práctica se utiliza un miembro por replica set para trabajar con los cinco servicios solicitados.

Repositorio



https://github.com/Kevin67-67/MongoDB-Cluster

Requisitos y archivos

Se utiliza Docker Desktop con motor de contenedores Linux, Docker Compose y PowerShell para ejecutar los comandos.

Archivo	Función
`docker-compose.yml`	Define cinco servicios, red, puertos, volúmenes y comandos de inicio
`scripts/01-config.js`	Inicializa el replica set configRS
`scripts/02-shard.js`	Inicializa el replica set de cada uno de los tres shards
`scripts/03-sharding.js`	Registra los shards, crea las colecciones e índices y fragmenta empleados
`scripts/04-scott.js`	Inserta los 4 departamentos y 14 empleados originales
`scripts/05-datos-adicionales.js`	Inserta 100 empleados adicionales con nombres y valores variados
`scripts/06-crud.js`	Inserta, consulta, actualiza y elimina un empleado temporal
PASOS.md	Explica la configuración inicial y la reanudación manual
README.md	Describe el proyecto e incluye la clonación y ejecución en otro equipo

El puerto publicado se cambió a 27030 porque 27017 ya estaba ocupado en Windows debido a anteriores prácticas.

Estructura de la base de datos Scott

Departamentos almacena deptno, dname y loc. Empleados almacena empno, ename, job, mgr, hiredate, sal, comm y deptno. MongoDB agrega _id y las fechas se almacenan como BSON Date. mgr y comm admiten valores nulos según el documento.

Ejemplo de empleado:

```
{
  empno: 7369, ename: "SMITH", job: "CLERK",
  mgr: 7902, hiredate: ISODate("1980-12-17"),
  sal: 800, comm: null, deptno: 20
}
```

La carga conserva los 4 departamentos y 14 empleados originales. Se añaden 100 empleados de ejemplo.

Elección de la shard key

Para la colección scott.empleados elegimos { empno: "hashed" } como shard key porque empno es el número que identifica a cada empleado y permite buscarlo directamente, cuando se usa hash, los empleados pueden repartirse entre los shards aunque sus números sean consecutivos.

Decidimos no elegir deptno porque solo tenemos cuatro departamentos y algunos tienen más empleados que otros. Usar hash sobre este campo no resolvería esa limitación, ya que seguirían siendo pocos valores diferentes. Esta elección funciona para buscar empleados por su número, entonces si la mayoría de las consultas fueran por departamento, tendríamos que considerar otra clave.

Cuando se busca un valor exacto de empno, mongos puede dirigir la consulta al shard correspondiente. En cambio, una búsqueda solo por deptno puede necesitar consultar todos los shards.

Aplicación y comandos

Primera configuración de servidores y replica sets

- `docker compose up -d configsvr shard1 shard2 shard3`

Después de dejar tiempo para que MongoDB inicie:

- `docker compose exec -T configsvr mongosh --quiet --file /scripts/01-config.js`
- `docker compose exec -T shard1 mongosh --quiet --file /scripts/02-shard.js`
- `docker compose exec -T shard2 mongosh --quiet --file /scripts/02-shard.js`
- `docker compose exec -T shard3 mongosh --quiet --file /scripts/02-shard.js`

Router, registro de shards y fragmentación

- `docker compose up -d mongos`

Después del inicio del router:

- `docker compose exec -T mongos mongosh --quiet --file /scripts/03-sharding.js`

Carga de Scott y empleados adicionales

- `docker compose exec -T mongos mongosh --quiet --file /scripts/04-scott.js`
- `docker compose exec -T mongos mongosh --quiet --file /scripts/05-datos-adicionales.js`

Ambas cargas corresponden a la primera ejecución. No se repiten al reiniciar.

Consultas

Desde PowerShell:

- `docker compose exec mongos mongosh`

Dentro de mongosh:

- `use scott`
- `sh.status()`
- `db.departamentos.countDocuments({})`
- `db.empleados.countDocuments({})`
- `db.empleados.countDocuments({ empno: { $gte: 8000, $lte: 8099 } })`
- `db.empleados.getShardDistribution()`
- ```
  db.getSiblingDB("admin").aggregate([
    { $shardedDataDistribution: {} },
    { $match: { ns: "scott.empleados" } }
  ]).toArray()
  ```
- `exit`

Detener y reanudar sin recargar

Para detener desde la terminal del sistema:

- `docker compose stop`

Para reanudar contenedores existentes:

- `docker compose start`
  ```

Evidencias de ejecución y resultados

```
PS C:\Users\kevin\Escuela\Bases\Parcial 3\Actividad 1> docker compose up -d configsvr shard1 shard2 sh
ard3
[+] up 20/20
 ✔ Image mongo:8.0.16                    Pulled                                                 24.6s
 ✔ Volume actividad1-scott_shard3-data   Created                                                 0.0s
 ✔ Volume actividad1-scott_config-data   Created                                                 0.0s
 ✔ Volume actividad1-scott_shard1-data   Created                                                 0.0s
 ✔ Volume actividad1-scott_shard2-data   Created                                                 0.0s
 ✔ Network actividad1-scott_mongo-net    Created                                                 0.1s
 ✔ Container actividad1-scott-shard3-1   Started                                                 1.7s
 ✔ Container actividad1-scott-configsvr-1 Started                                                0.6s
 ✔ Container actividad1-scott-shard2-1   Started                                                 0.9s
 ✔ Container actividad1-scott-shard1-1   Started                                                 1.1s
```

Iniciar servidores

```
PS C:\Users\kevin\Escuela\Bases\Parcial 3\Actividad 1> docker compose exec -T shard1 mongosh --quiet -
-file /scripts/02-shard.js
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793233786, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793233786, i: 1 })
}
PS C:\Users\kevin\Escuela\Bases\Parcial 3\Actividad 1> docker compose exec -T shard2 mongosh --quiet -
-file /scripts/02-shard.js
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793233789, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793233789, i: 1 })
}
PS C:\Users\kevin\Escuela\Bases\Parcial 3\Actividad 1> docker compose exec -T shard3 mongosh --quiet -
-file /scripts/02-shard.js
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793233800, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793233800, i: 1 })
}
```

Shards

```
PS C:\Users\kevin\Escuela\Bases\Parcial 3\Actividad 1> docker compose up -d mongos
[+] up 1/1
 ✔Container actividad1-scott-mongos-1 Started                                    0.5s
PS C:\Users\kevin\Escuela\Bases\Parcial 3\Actividad 1> docker compose exec -T mongos mongosh --quiet --
-file /scripts/03-sharding.js
{
  shardAdded: 'shard1RS',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793234223, i: 20 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793234223, i: 20 })
}
{
  shardAdded: 'shard2RS',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793234223, i: 43 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793234223, i: 37 })
}
{
```

Router y sharding

```
PS C:\Users\kevin\Escuela\Bases\Parcial 3\Actividad 1> docker compose exec -T mongos mongosh --quiet --
-file /scripts/04-scott.js
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ae29770dc6aa5a0b29dc29d'),
    '1': ObjectId('6ae29770dc6aa5a0b29dc29e'),
    '2': ObjectId('6ae29770dc6aa5a0b29dc29f'),
    '3': ObjectId('6ae29770dc6aa5a0b29dc2a0')
  }
}
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ae29770dc6aa5a0b29dc2a1'),
    '1': ObjectId('6ae29770dc6aa5a0b29dc2a2'),
    '2': ObjectId('6ae29770dc6aa5a0b29dc2a3'),
    '3': ObjectId('6ae29770dc6aa5a0b29dc2a4'),
    '4': ObjectId('6ae29770dc6aa5a0b29dc2a5'),
    '5': ObjectId('6ae29770dc6aa5a0b29dc2a6'),
    '6': ObjectId('6ae29770dc6aa5a0b29dc2a7'),
    '7': ObjectId('6ae29770dc6aa5a0b29dc2a8'),
    '8': ObjectId('6ae29770dc6aa5a0b29dc2a9'),
    '9': ObjectId('6ae29770dc6aa5a0b29dc2aa'),
    '10': ObjectId('6ae29770dc6aa5a0b29dc2ab'),
    '11': ObjectId('6ae29770dc6aa5a0b29dc2ac'),
    '12': ObjectId('6ae29770dc6aa5a0b29dc2ad'),
    '13': ObjectId('6ae29770dc6aa5a0b29dc2ae')
  }
}
PS C:\Users\kevin\Escuela\Bases\Parcial 3\Actividad 1> docker compose exec -T mongos mongosh --quiet --
-file /scripts/05-datos-adicionales.js
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ae297755620f0b4009dc29d'),
    '1': ObjectId('6ae297755620f0b4009dc29e'),
    '2': ObjectId('6ae297755620f0b4009dc29f'),
    '3': ObjectId('6ae297755620f0b4009dc2a0'),
    '4': ObjectId('6ae297755620f0b4009dc2a1'),
    '5': ObjectId('6ae297755620f0b4009dc2a2'),
    '6': ObjectId('6ae297755620f0b4009dc2a3'),
    '7': ObjectId('6ae297755620f0b4009dc2a4'),
    '8': ObjectId('6ae297755620f0b4009dc2a5'),
    '9': ObjectId('6ae297755620f0b4009dc2a6'),
    '10': ObjectId('6ae297755620f0b4009dc2a7'),
    '11': ObjectId('6ae297755620f0b4009dc2a8'),
    '12': ObjectId('6ae297755620f0b4009dc2a9'),
    '13': ObjectId('6ae297755620f0b4009dc2aa'),
    '14': ObjectId('6ae297755620f0b4009dc2ab'),
```

Carga de Scott y datos adicionales

```
PS C:\Users\kevin\Escuela\Bases\Parcial 3\Actividad 1> docker compose exec -T mongos mongosh --quiet --file /scripts/06-crud.js
CREATE: insertar empleado temporal
{
  acknowledged: true,
  insertedId: ObjectId('6ae29840c0264357729dc29d')
}
READ: consultar empleado por empno
{
  _id: ObjectId('6ae29840c0264357729dc29d'),
  empno: 999999,
  ename: 'EMPLEADO_CRUD',
  job: 'CLERK',
  mgr: 7839,
  hiredate: ISODate('2026-10-28T00:00:00.000Z'),
  sal: 1500,
  comm: null,
  deptno: 20,
  origen: 'crud'
}
UPDATE: cambiar salario y puesto
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
{
  _id: ObjectId('6ae29840c0264357729dc29d'),
  empno: 999999,
  ename: 'EMPLEADO_CRUD',
  job: 'ANALYST',
  mgr: 7839,
  hiredate: ISODate('2026-10-28T00:00:00.000Z'),
  sal: 1800,
  comm: null,
  deptno: 20,
  origen: 'crud'
}
DELETE: eliminar exclusivamente el empleado temporal
{
  acknowledged: true,
  deletedCount: 1
}
{
  despuesEliminar: null
}
```

CRUD

```
 C:\Users\kevin\Escuela\Bases\Parcial 3\Actividad 1> docker compose stop
+] stop 5/5
✔Container actividad1-scott-shard2-1    Stopped                                 1.3s
✔Container actividad1-scott-mongos-1    Stopped                                 3.3s
✔Container actividad1-scott-shard3-1    Stopped                                 3.3s
✔Container actividad1-scott-shard1-1    Stopped                                 0.5s
✔Container actividad1-scott-configsvr-1 Stopped                                 0.4s
 C:\Users\kevin\Escuela\Bases\Parcial 3\Actividad 1> docker compose start
+] start 5/5
✔Container actividad1-scott-configsvr-1 Started                                 0.2s
✔Container actividad1-scott-mongos-1    Started                                 0.3s
✔Container actividad1-scott-shard1-1    Started                                 0.6s
✔Container actividad1-scott-shard2-1    Started                                 0.7s
✔Container actividad1-scott-shard3-1    Started                                 0.9s
```

```
[direct: mongos] scott> db.getSiblingDB("admin").aggregate([
...   { $shardedDataDistribution: {} },
...   { $match: { ns: "scott.empleados" } }
... ]).toArray()
[
  {
    ns: 'scott.empleados',
    shards: [
      {
        shardName: 'shard2RS',
        numOrphanedDocs: 0,
        numOwnedDocuments: 29,
        ownedSizeBytes: 3538,
        orphanedSizeBytes: 0
      },
      {
        shardName: 'shard1RS',
        numOrphanedDocs: 0,
        numOwnedDocuments: 50,
        ownedSizeBytes: 6150,
        orphanedSizeBytes: 0
      },
      {
        shardName: 'shard3RS',
        numOrphanedDocs: 0,
        numOwnedDocuments: 35,
        ownedSizeBytes: 4270,
        orphanedSizeBytes: 0
      }
    ]
  }
]
```

Distribución de shards

Conclusión

En esta práctica se implementó un clúster MongoDB sobre Docker con un servidor de configuración, tres shards y un router mongos, y luego se registraron los shards desde mongos. La base de datos Scott se organizó en departamentos, sin fragmentar, y empleados, fragmentada con la shard key { empno: "hashed" }, se hizo de esta manera ya que los departamentos son solo 4 y no se vería un gran cambio, mientras que los empleados son más de 100 y se puede observar de mejor manera como se trabaja con los shards.

La distribución lanzó 50, 29 y 35 documentos en shard1RS, shard2RS y shard3RS, con un total de 114. Aunque reparto no es perfectamente parejo, se puede esperar, ya que no son muchos datos; con un volumen mayor, el hash tiende a equilibrarse, y las operaciones CRUD ejecutadas desde mongos funcionaron correctamente, esto nos confirma que el router dirige cada operación al shard correspondiente sin que el cliente conozca dónde están los datos.

Referencias

MongoDB. *Deploy a Self-Managed Sharded Cluster*. https://www.mongodb.com/docs/manual/tutorial/deploy-shard-cluster/

MongoDB. *Hashed Sharding — MongoDB 8.0*. https://www.mongodb.com/docs/v8.0/core/hashed-sharding/

MongoDB. *shardCollection — MongoDB 8.0*. https://www.mongodb.com/docs/v8.0/reference/command/shardcollection/

MongoDB. *$shardedDataDistribution*. https://www.mongodb.com/docs/manual/reference/operator/aggregation/shardeddatadistribution/

Docker. *Docker Compose*. https://docs.docker.com/compose/

Kavaturu, U. *MongoDB Sharding Demo with Docker*. https://github.com/udaykirankavaturu/sharding-demo

Material de referencia de la actividad: BDA_SettingUpMongoDBCluster.pdf y BDA_MongoDBSharding.pdf.